

KRduinoUNO 开发板

用户手册

版本信息

版本	日期	修订者	说明
V0.1	2018/10/27	Mai	初次编写
V0.2	2019/05/22	Mai	修正规格参数
V1.0	2021/03/10	Mai	新增常见问题
V2.0	2022/07/16	Mai	新增程序示例
V2.1	2023/08/01	Mai	更新软件配图

声明：

本指导教程和配套程序仅在开发和学习中参考，不得用于商业用途，如需转载或引用，请保留版权声明和出处。

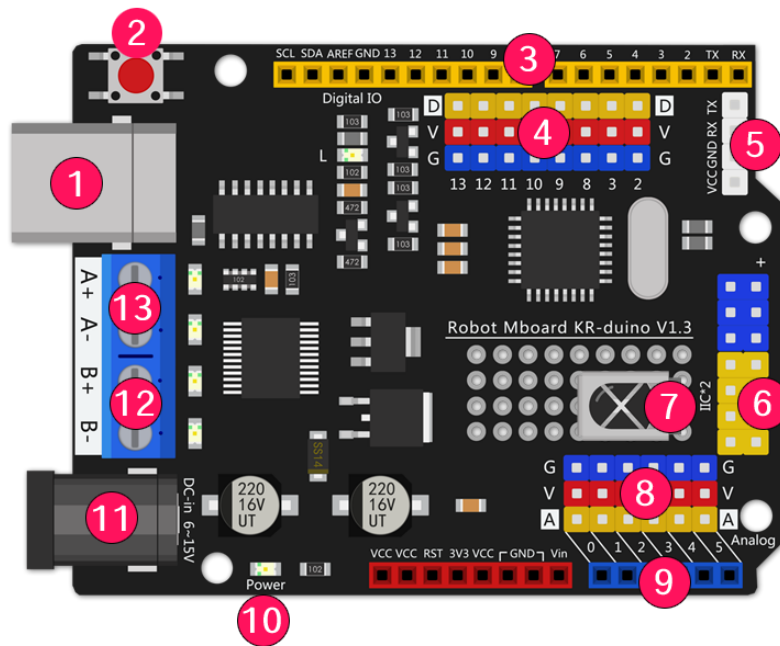
自行扩展搭接导致不良故障，本公司不负任何责任。产品不定时升级，所有更改恕不另行通知，本公司有最终解释权。

科睿电子科技长期致力于改善其产品的质量和可靠性。但是，一般的电子器件所固有的电子敏感及物理损坏特性可能会造成器件产生故障。因此，消费者有责任依照安全标准使用科睿电子科技的产品，并且避免由于科睿电子科技的产品故障所造成的人身伤害或财产损失。设计者应当参考科睿电子科技提供的最新的产品操作范围进行产品设计。详细的使用注意事项和工作条件请参考手册参数。

一、概述

KRduino UNO 是由科睿电子科技基于原版 Arduino UNO R3 开发板改进优化,在完全兼容原版 UNO 的基础上更加简便易用。相较于其他版本 UNO, 增加并集成了可以同时驱动两路直流减速电机的电机驱动芯片和额外的 IO 扩展口、红外接收头、串口接口等。多功能的设计,力求让板卡有更多用武之地,让精力从烦扰的接线中解脱,帮助您轻松快速实现智能车、船或其他有趣的功能。

二、功能介绍



- ① USB-B 座, 用于连接电脑上传程序
- ② RST 复位按键, 开发板内的程序重新从头开始运行
- ③ Arduino UNO 原生接口, D0-D13 数字引脚、AD 参考以及 IIC 接口
- ④ 额外引出排针, 以-+s 方式排列。黄色排针 (Digital 数字引脚) 从右往左分别为 D2、D3、D8-D13 (与③中的引脚是连在一起的), 红色排针 (VCC) 为 5V 电源正极, 蓝色排针 (GND) 为 5V 电源负极
- ⑤ 额外增加带电源的串口 (与③中的 D0、D1 引脚是连在一起的) 可以方便连接蓝牙、WIFI 等串口通信的模块
- ⑥ 两路带电源的 IIC 接口, 用于需要使用多个 IIC 模块通信
- ⑦ 38KHz 红外接收头 (与 A3 引脚是连在一起的), 满足大多数无线遥控场景
- ⑧ 额外引出的排针, 以-+s 方式排列, 黄色排针 (Analog 模拟输入引脚) 从右往左依次为 A5-A0 (与⑨中的引脚是连在一起的), 红色排针 (VCC) 为 5V 电源正极, 蓝色排针 (GND) 为 5V 电源负极
- ⑨ Arduino UNO 原生接口, A0-A5 模拟输入引脚
- ⑩ 电源指示灯
- ⑪ DC 输入, 支持 6-15V 直流电源供电
- ⑫ B 组电机, D4 控制电机转动方向(DIR), D5 控制电机转动速度(PWM)
- ⑬ A 组电机, D7 控制电机转动方向(DIR), D6 控制电机转动速度(PWM)

三、规格参数

电气参数	
主控芯片	Atmel Mage32P
DC 输入电压	直流 6-15V
USB 输入电压	直流 5V
A 组电机驱动电压	约等于外部供电电压（USB 口或 DC 口的电压）
A 组电机驱动电流	持续电流:1.2A 启动峰值电流:2A
B 组电机驱动电压	约等于外部供电电压（USB 口或 DC 口的电压）
B 组电机驱动电流	持续电流:1.2A 启动峰值电流:2A
红外接收频率	38KHz
+s 排针额定电压	5V 直流
+s 排针额定电流	DC 口供电:Max 500mA USB 供电:与供电设备一致

机械参数	
最大轮廓尺寸	75mm*53.5mm*14.5mm（长*宽*高）
安装孔直径	Φ3
USB 座型号	USB-B 母座
DC 座型号	DC5521
排针/座 针/孔间距	2.54mm
电机接口间距	5.08mm
重量	30g
工作温度	0-40℃（推荐）

四、注意事项

- 1、连接电机时，一个电机的两个接线要接在同一组，即 A 组或者 B 组的+-上。
- 2、驱动电机时，电机的堵转电流不建议超过 1.2A，不得超过 2A。
- 3、用 DC 口外接电源供电时，外接的模块总电流不要超过 500mA，USB 供电或单独给 5V 引脚上供电无此限制。
- 4、初次使用，用 USB 数据线与电脑连接后电脑会自动安装驱动，没有则参考[手动安装教程](#)。
- 5、除需要串口通信的模块外，尽量不要使用 D0 和 D1，有可能会导致无法正常上传程序。
- 6、如 D0 或 D1 外接了模块，在上传程序失败时请拔掉连接在 D0、D1 的线再上传试试。
- 7、因红外接收头连接了 A3 引脚，可优先使用其他空余引脚（无红外信号不影响）。
- 8、使用红外遥控器时，红外遥控器的发射 LED 对着红外接收头黑色接收窗。

五、常见问题

Q:红外遥控器解码 库的问题

A:如编译信息提示没有找打 IRomote.h 则需要手动添加库，参考[视频教程传送门](#)。注意库版本要与程序的注释一致。

Q:红外遥控器使用问题，无法遥控

A:查看红外遥控器是否正常，注意有些手机主摄像头有红外滤光片需要切换镜头，红外遥控是红外光通信，所以需要对着接收头不能有遮挡。

Q:无法上传程序，提示 stk500_recv():xxxx

A:属于串口通信异常，检查一下 USB 插入电脑后有无 COM 口，并参考注意事项中的 4、5、6 条进行自检自查。

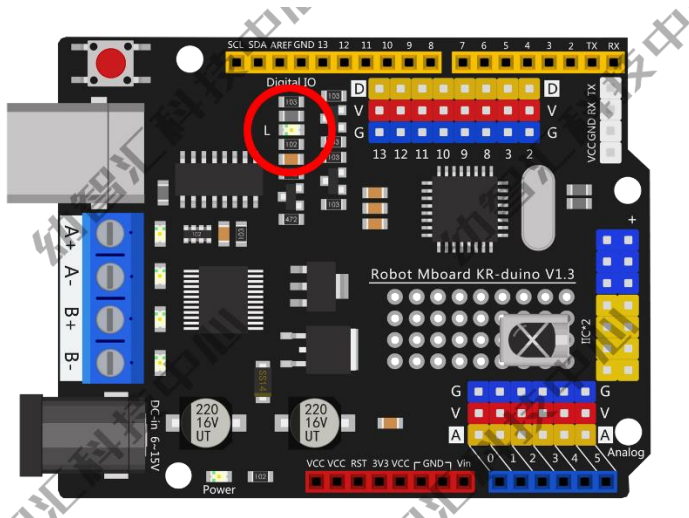
Q:驱动一个或两个 SG90 舵机运行正常，同时驱动两个以上 SG90 舵机，舵机无法正常工作。

A: 考虑属于电流过载导致开发板无法正常工作，参考注意事项中的 3 做合适改进。

六、程序示例

1、开关量输出模块示例-

<1>LED（使板载 LED 间隔 1 秒闪烁，即亮一秒灭一秒）



LED 位置示意图

①Arduino 示例（可直接复制）

`const int LED=13;` //D13 可以控制板子上的 LED 灯

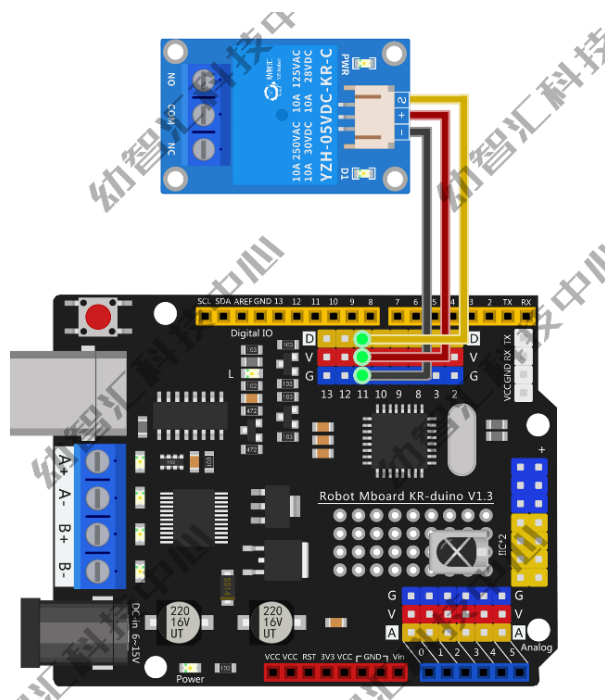
```
void setup(){
    pinMode(LED, OUTPUT);    //板载 LED 引脚是 D13 设置为输出
}

void loop(){
    digitalWrite(LED,HIGH);    //LED 亮
    delay(1000);                //延时等待 1s
    digitalWrite(LED,LOW);     //LED 灭
    delay(1000);                //延时等待 1s
}
```

②Mixly 2.0 rc3 版本示例



<2>继电器（通一秒，断一秒）



接线示意图

①Arduino 示例（可直接复制）

```
const int JDQ=11; //用 D11 控制继电器通断

void setup(){
    pinMode(JDQ, OUTPUT);    //引脚 D11 设置为输出
}

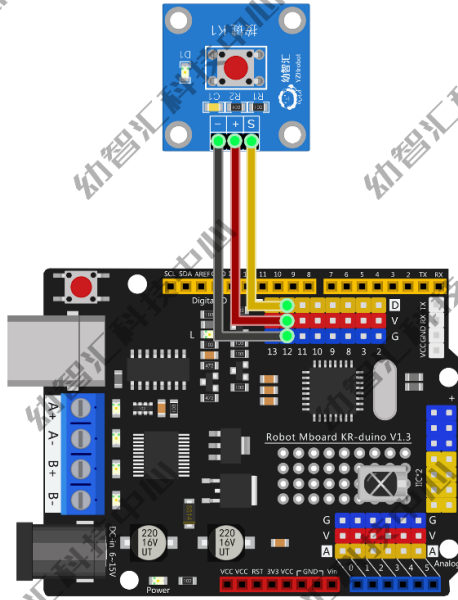
void loop(){
    digitalWrite(JDQ,HIGH);    //继电器通
    delay(1000);                //延时等待 1s
    digitalWrite(JDQ,LOW);    //继电器断
    delay(1000);                //延时等待 1s
}
```

②Mixly 2.0 rc3 版本示例



3、开关量输入模块示例-

<1>按键（按下按键 LED 亮，松开按键 LED 灭）



独立按键连接示意图

①Arduino 示例（可直接复制）

```
const int KEY=12; //按键模块接在 D12
const int LED=13; //D13 可以控制板子上的 LED 灯

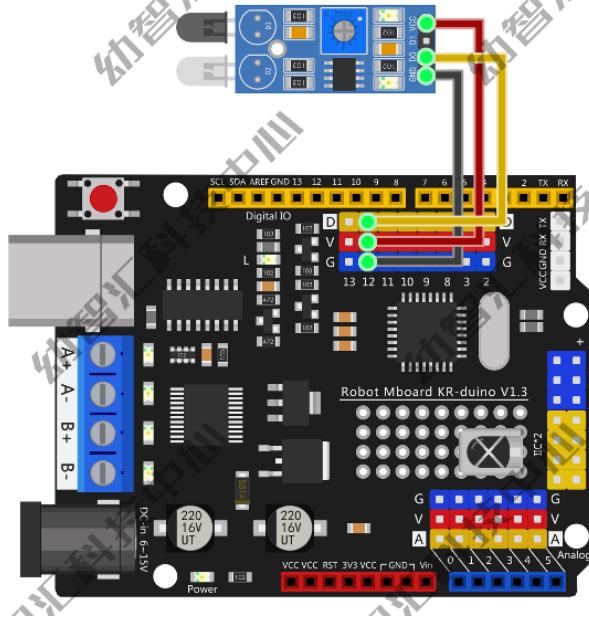
void setup(){
    pinMode(KEY, INPUT);    //检测按键需要设置为输入
    pinMode(LED, OUTPUT);   //板载 LED 引脚是 D13 设置为输出
}

void loop(){
    if (digitalRead(KEY)==HIGH) {    //如果按键按下
        digitalWrite(LED,HIGH);     //LED 亮
    } else{                          //如果按键没有按下
        digitalWrite(LED,LOW);      //LED 灭
    }
}
```

②Mixly 2.0 rc3 版本示例



<2>红外感应（没有信号 LED 慢闪；有信号 LED 快闪）



接线示意图

Arduino 示例（可直接复制）

```
const int IR=12; //红外模块信号输出接在 D12
const int LED=13; //D13 可以控制板子上的 LED 灯

void setup(){
    pinMode(IR, INPUT);          //检测信号需要设置为输入
    pinMode(LED, OUTPUT);        //板载 LED 引脚是 D13 设置为输出
}

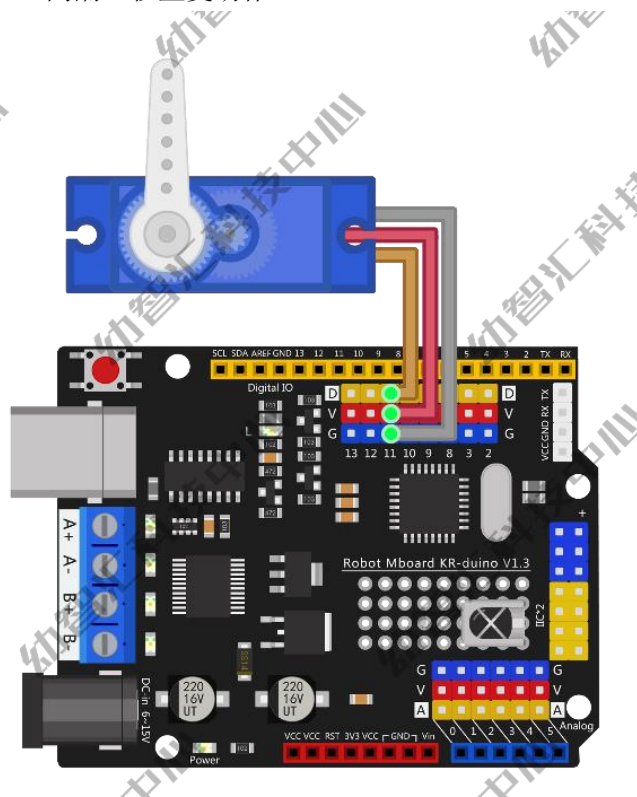
void loop(){
    if (digitalRead(IR)==HIGH) { //判断是否有信号
        digitalWrite(LED,HIGH); //LED 亮
        delay(100);              //延时等待 100ms
        digitalWrite(LED,LOW);   //LED 灭
        delay(100);              //延时等待 100ms
    }
    else                          //如果没有信号
    {
        digitalWrite(LED,HIGH); //LED 亮
        delay(300);              //延时等待 300ms
        digitalWrite(LED,LOW);   //LED 灭
        delay(300);              //延时等待 300ms
    }
}
```

Mixly 2.0 rc3 版本示例



7、模拟量输出模块示例

<1>SG90 舵机（0° -90° 间隔 1 秒重复动作）



接线示意图

Arduino 示例（可直接复制）

```
#include <Servo.h>           //引用舵机库
Servo servo_11;               //声明 Servo 对象为 servo_11

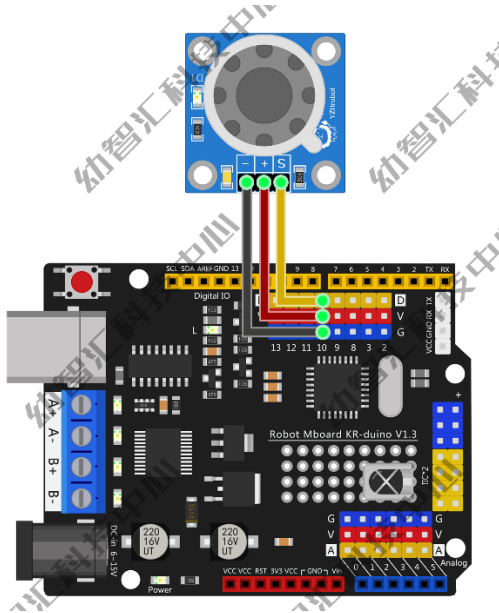
void setup(){
    servo_11.attach(11);      //初始化舵机控制口为 D11
}

void loop(){
    servo_11.write(0);         //控制舵机转动到 0 度
    delay(1000);               //延时等待 1S
    servo_11.write(90);        //控制舵机转动到 90 度
    delay(1000);               //延时等待 1S
}
```

Mixly 2.0 rc3 版本示例



<2>无源蜂鸣器（生日快乐歌）



接线示意图

Arduino 示例（可直接复制）

```
void setup(){
    pinMode(10, OUTPUT); //无源蜂鸣器接到 D10 引脚 设置 D10 引脚为输出
}

void NewTone(int pin,int hz,int time) //音符发声函数 pin 是频率输出引脚 hz
是频率 time 是发声时间
{
    tone(pin,hz); //发声
    delay(time); //持续时间
    noTone(pin); //停止发声
}

void loop(){
    NewTone(10,196,375); //D10 引脚 196Hz 频率 发声 375ms
    NewTone(10,196,375);
    NewTone(10,220,750);
    NewTone(10,196,750);
    NewTone(10,262,750);
    NewTone(10,247,1000);
    delay(1000);
}
```

Mixly 2.0 rc3 版本示例



播放声音(无定时器)

管脚 #

10

频率

NOTE_G3

持续时间

375

毫秒

播放声音(无定时器)

管脚 #

10

频率

NOTE_G3

持续时间

375

毫秒

播放声音(无定时器)

管脚 #

10

频率

NOTE_A3

持续时间

750

毫秒

播放声音(无定时器)

管脚 #

10

频率

NOTE_G3

持续时间

750

毫秒

播放声音(无定时器)

管脚 #

10

频率

NOTE_C4

持续时间

750

毫秒

播放声音(无定时器)

管脚 #

10

频率

NOTE_B3

持续时间

1000

毫秒

延时

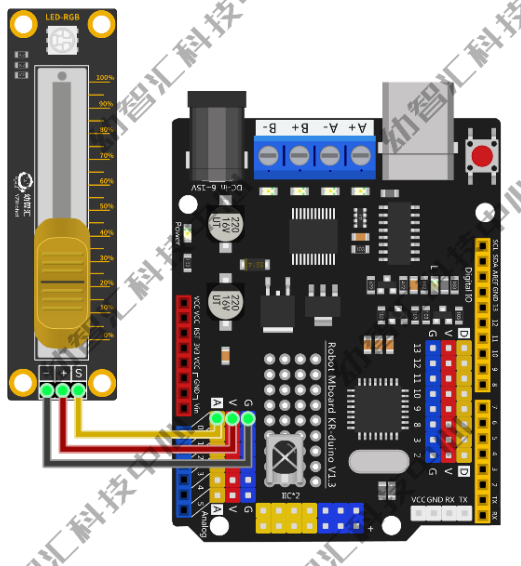
毫秒

1000

KRduinoUNO 开发板

6、模拟量输入模块示例

<1>电位器（通过电位器改变板载 LED 闪烁频率）



接线示意图

Arduino 示例（可直接复制）

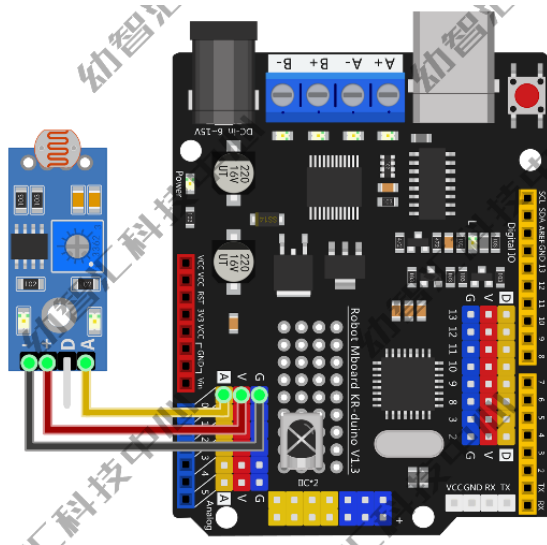
```
const int RES=A0; //滑杆模块接在 A0
const int LED=13; //D13 可以控制板子上的 LED 灯
int time=300; //延时等待的时间
void setup(){
    pinMode(LED, OUTPUT); //板载 LED 引脚是 D13 设置为输出
}
void loop(){
    time=analogRead(RES); //读取滑杆模块的 AD 数值（0~1023）
    time=map(time,0,1023,1,500); //将 0~1023 同比映射为 1~500（类似缩放）
    digitalWrite(LED,HIGH); //LED 亮
    delay(time); //延时等待
    digitalWrite(LED,LOW); //LED 灭
    delay(time); //延时等待
}
```

Mixly 2.0 rc3 版本示例



KRduinoUNO 开发板

<2>光敏（光线超过一定值后板载 LED 亮）



接线示意图

Arduino 示例（可直接复制）

```
const int RES=A0; //光敏模块接在 A0
const int LED=13; //D13 可以控制板子上的 LED 灯
int value=0;      //存放 AD 数值
void setup(){
    pinMode(LED, OUTPUT);    //板载 LED 引脚是 D13 设置为输出
}
void loop(){
    value=analogRead(RES);    //读取滑杆模块的 AD 数值（0~1023）
    if(value>500){            //判断数值是否大于 500
        digitalWrite(LED,HIGH); //条件成立（value 大于 500）LED 亮
    }else{
        digitalWrite(LED,LOW);  //条件不成立（value 小于等于 500）LED 灭
    }
}
```

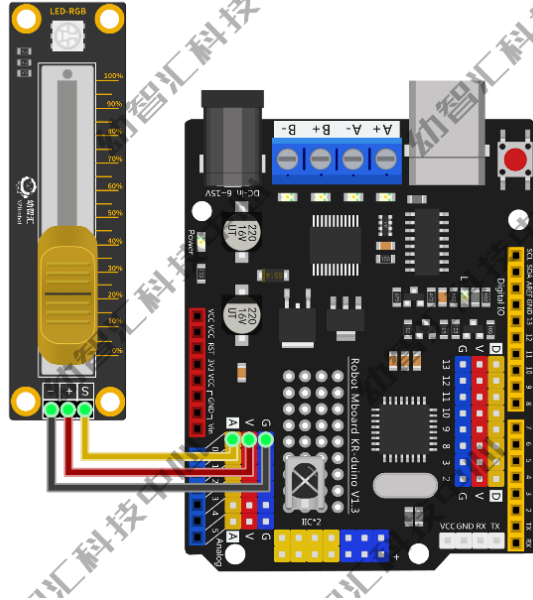
Mixly 2.0 rc3 版本示例



KRduinoUNO 开发板

8、串并行通信模块示例

<1>串口通信（输出模拟量数值）



接线示意图

Arduino 示例（可直接复制）

```
const int RES=A0; //滑杆模块接在 A0
int value=0;      //存放数值

void setup(){
    Serial.begin(9600); //串口波特率 9600
}
void loop(){
    value=analogRead(RES); //读取滑杆模块的 AD 数值（0~1023）
    Serial.print("value="); //串口输出英文字符 不换行
    Serial.println(value); //串口输出 value 的值 并换行
    delay(100); //延时等待 100 毫秒
}
```

Mixly 2.0 rc3 版本示例



KRduinoUNO 开发板

<2>串口通信（串口接收命令并执行 可用于蓝牙遥控哦）

Arduino 示例（可直接复制）

String recv; //声明字符串变量 用于缓存接收到的数据

```
void serialEvent() { //串口事件服务程序
    recv = recv + Serial.readString(); //将新的字符串追加到 recv 字符串后面
}

void setup(){
    Serial.begin(9600); //串口波特率为 9600
    pinMode(13, OUTPUT); //D13 为板载 LED 控制引脚
}

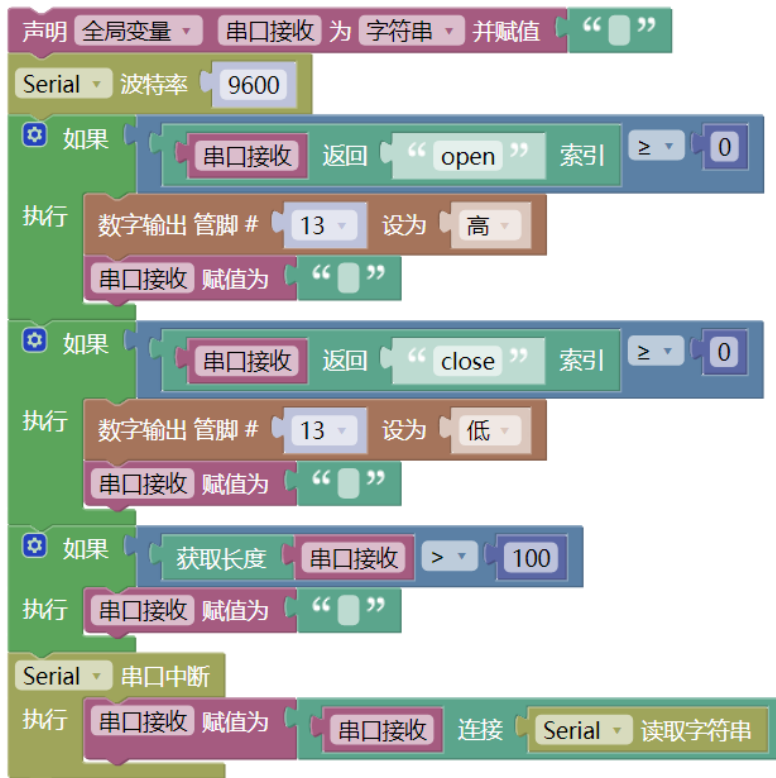
void loop(){
    if (recv.indexOf("open") >= 0) { //判断 recv 字符串（缓存区）内是否包含
open 这个字符串
        digitalWrite(13,HIGH); //包含 LED 灯亮
        recv = ""; //清空缓存
    }

    if (recv.indexOf("close") >= 0) { //判断 recv 字符串（缓存区）内是否包含
close 这个字符串
        digitalWrite(13,LOW); //包含 LED 灯灭
        recv = ""; //清空缓存
    }

    if (recv.length() > 100) { //判断缓存区是否过大
        recv = ""; //过大说明接收到了许多用不到的信息 清空
缓存
    }
}
```

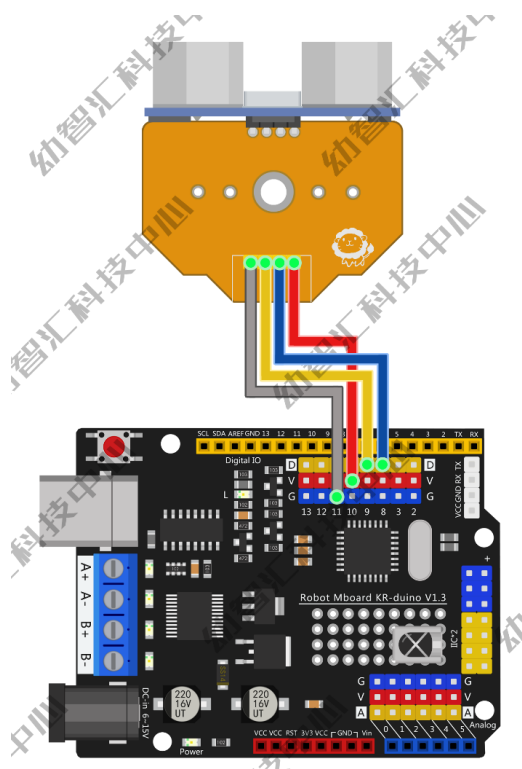
KRduinoUNO 开发板

Mixly 2.0 rc3 版本示例



5、开关量输入输出模块示例

超声波模块（小于等于 15cm 板载 LED 亮，大于 15cm 板载 LED 灭，并通过串口输出距离值）



接线示意图

Arduino 示例（可直接复制）

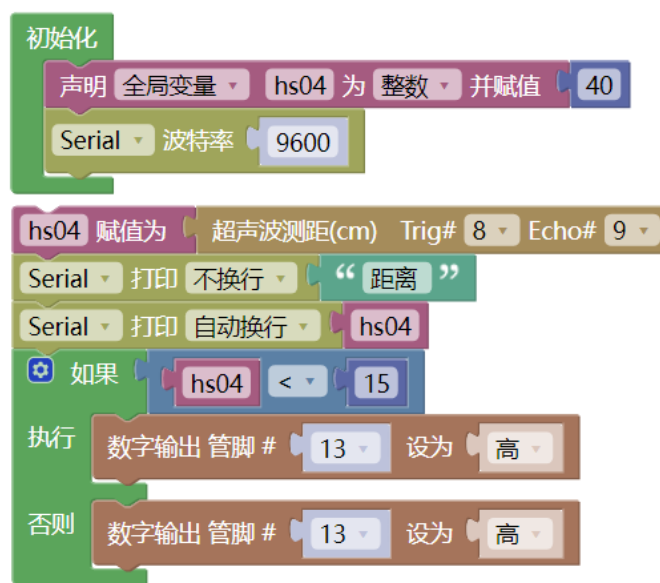
```
const int LED=13;           //D13 引脚可以控制板载 LED
volatile int hs04;           //用于存放测到的距离值

float checkdistance_8_9() {           //超声波测距函数
    digitalWrite(8, LOW);
    delayMicroseconds(2);
    digitalWrite(8, HIGH);
    delayMicroseconds(10);
    digitalWrite(8, LOW);           //发送两个脉冲
    float distance = pulseIn(9, HIGH) / 58.00;           //计时 直到 D9 变为高电平后
    //（超声波回波到达模块） 用时间/58 得到距离
    delay(30);           //延时等待 30ms
    return distance;           //返回距离值
}

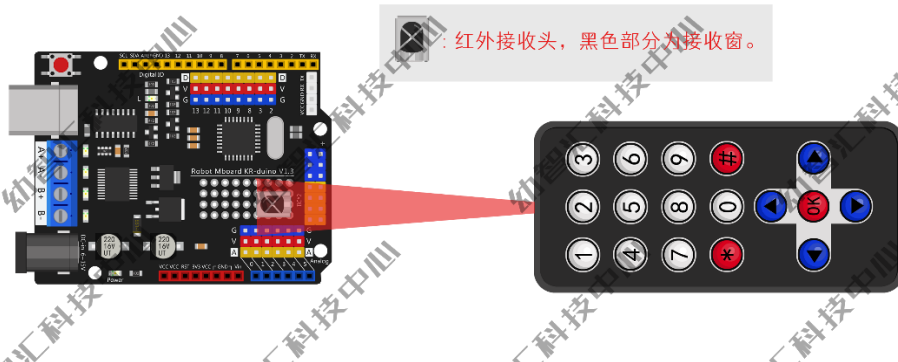
void setup(){
    hs04 = 40;           //变量赋初值
    Serial.begin(9600);           //串口波特率为 9600
    pinMode(LED, OUTPUT);           //D13 引脚设置为输出 用于控制 LED
    pinMode(8, OUTPUT);           //超声波模块的 TRGE 引脚 用于触发模块发射超声波
    pinMode(9, INPUT);           //超声波模块的 ECHO 引脚 用于检测超声波回波是否到
    达
    delay(200);
}

void loop()
{
    hs04 = checkdistance_8_9(); //控制超声波测量一次距离 并把距离数值赋给 hs04
    变量
    Serial.print("距离 ");
    Serial.println(hs04);           //通过串口输出测到的距离值
    if (hs04 < 15) {           //判断距离值是否小于 15
        digitalWrite(LED, HIGH); //距离值小于 15 板载 LED 灯亮
    }
    else {
        digitalWrite(LED, LOW); //距离值大于等于 15 板载 LED 灯灭
    }
}
```

Mixly 2.0 rc3 版本示例



2、红外接收示例（按 1、2、3 改变 D13 控制的板载 LED 的闪烁速度）



操作示意

① Arduino 示例（可直接复制，注意是否已添加 **IRremote** 库）

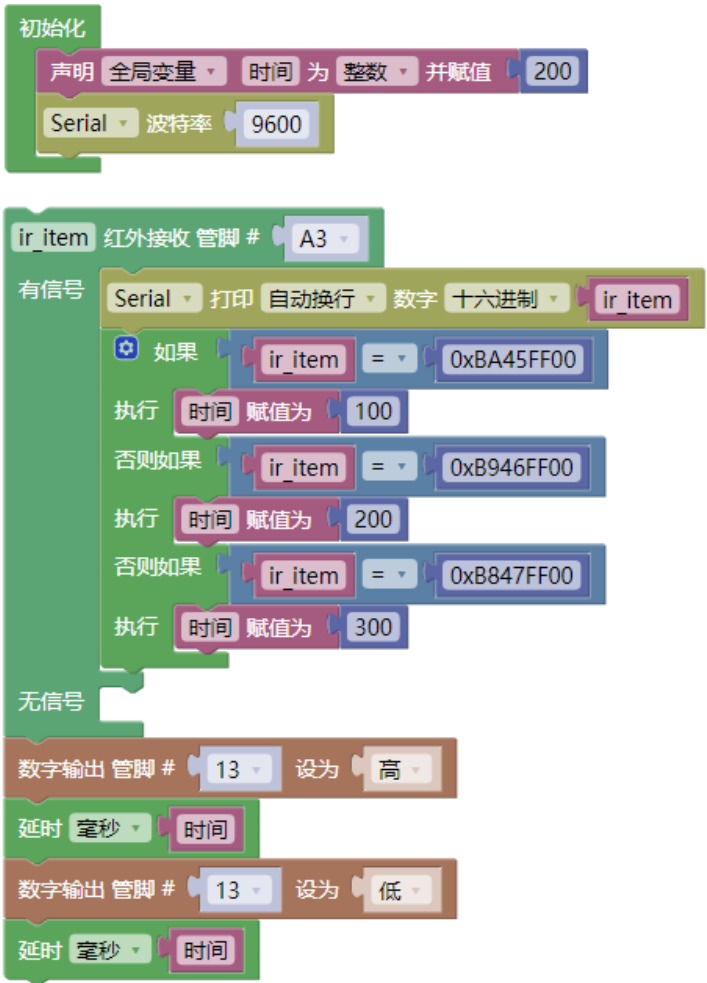
```
#include <IRremote.h> //红外库版本 3.7.1
volatile int time;      //闪烁时间间隔
IRrecv irrecv_A3(A3); //设定红外接收引脚为 A3

void setup(){
    time = 200;          //赋初值 200ms
    Serial.begin(9600);   //初始化串口 波特率 9600
    irrecv_A3.enableIRIn(); //使能红外接收引脚
    pinMode(13, OUTPUT);  //板载 LED 引脚是 D13 设置为输出
}

void loop(){
    if (irrecv_A3.decode()) { //如果成功解码
        struct IRData *pIrData = &irrecv_A3.decodedIRData; //准备读取解码信息
        long ir_item = pIrData->decodedRawData; //赋值给 ir_item
        Serial.println(ir_item, HEX); //串口发送编码
        irrecv_A3.resume(); //复位 便于下次接收
        if (ir_item == 0xBA45FF00) { //如果是按键 1
            time = 100; //LED 闪烁间隔 100ms
        } else if (ir_item == 0xB946FF00) { //如果是按键 2
            time = 200; //LED 闪烁间隔 200ms
        } else if (ir_item == 0xB847FF00) { //如果是按键 3
            time = 300; //LED 闪烁间隔 300ms
        }
    } else {}
    digitalWrite(13,HIGH); //LED 亮
    delay(time); //延时等待
    digitalWrite(13,LOW); //LED 灭
    delay(time); //延时等待
}
```



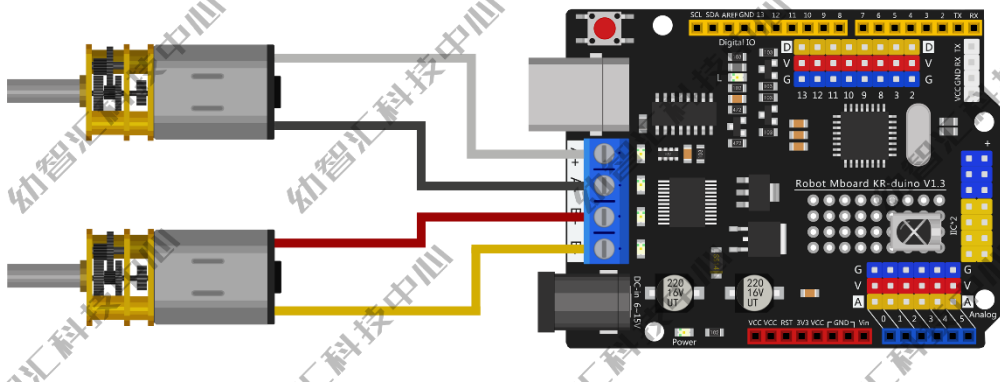
②Mixly 2.0 rc3 版本示例



③红外遥控器键值（针对 Arduino IDE 红外库 3.7.1 版本和 Mixly 2.0 rc3 版本）

【1】 0xBA45FF00	【2】 0xB946FF00	【3】 0xB847FF00
【4】 0xBB44FF00	【5】 0xBF40FF00	【6】 0xBC43FF00
【7】 0xF807FF00	【8】 0xEA15FF00	【9】 0xF609FF00
【*】 0xE916FF00	【0】 0xE619FF00	【#】 0xF20DFF00
	【↑】 0xE718FF00	
【←】 0xF708FF00	【OK】 0xE31CFF00	【→】 0xA55AFF00
	【↓】 0xAD52FF00	

1、驱动电机示例（AB 两组电机同时正转 1 秒再同时反转 1 秒，依此循环）



参考接线图

① Arduino 示例 1 （可直接复制）

```
//A 组电机驱动所需要的引脚
int A_PWM = 6; //控制速度
int A_DIR = 7; //控制方向
//B 组电机驱动所需要的引脚
int B_PWM = 5; //控制速度
int B_DIR = 4; //控制方向

//初始化
void setup(){
    pinMode(A_DIR, OUTPUT);
    pinMode(A_PWM, OUTPUT);
    pinMode(B_DIR, OUTPUT);
    pinMode(B_PWM, OUTPUT); //全部都设置为输出
}
//主循环
void loop(){
    digitalWrite(A_DIR, HIGH); //A 电机正转（默认 HIGH 为正转）
    analogWrite(A_PWM, 125); //A 电机速度值 125 （范围 0-255）
    digitalWrite(B_DIR, HIGH); //B 电机正转（默认 HIGH 为正转）
    analogWrite(B_PWM, 125); //B 电机速度值 125 （范围 0-255）
    delay(1000); //延时等待 1000ms
    digitalWrite(A_DIR, LOW); //A 电机反转（默认 LOW 为反转）
    analogWrite(A_PWM, 80); //A 电机速度值 80 （范围 0-255）
    digitalWrite(B_DIR, LOW); //B 电机反转（默认 LOW 为反转）
    analogWrite(B_PWM, 80); //B 电机速度值 80 （范围 0-255）
    delay(1000); //延时等待 1000ms
}
```

KRduinoUNO 开发板

② Arduino 示例 2（可直接复制）

```
//A 组电机驱动所需要的引脚
int A_PWM = 6; //控制速度
int A_DIR = 7; //控制方向

//B 组电机驱动所需要的引脚
int B_PWM = 5; //控制速度
int B_DIR = 4; //控制方向

//初始化
void setup(){
    pinMode(A_DIR, OUTPUT);
    pinMode(A_PWM, OUTPUT);
    pinMode(B_DIR, OUTPUT);
    pinMode(B_PWM, OUTPUT); //全部都设置为输出
}

//A 组电机驱动控制函数
void A_Motor(int dir,int speed)
{
    digitalWrite(A_DIR, dir);
    analogWrite(A_PWM, speed);
}

//B 组电机驱动控制函数
void B_Motor(int dir,int speed)
{
    digitalWrite(B_DIR, dir);
    analogWrite(B_PWM, speed);
}

//主循环
void loop(){
    A_Motor(HIGH,125); //A 电机正转(默认 HIGH 为正转),速度值 125 (范围 0-255)
    B_Motor(HIGH,125); //B 电机正转(默认 HIGH 为正转),速度值 125(范围 0-255)
    delay(1000);      //延时等待 1000ms
    A_Motor(LOW,80);  //A 电机反转(默认 LOW 为反转),速度值 80 (范围 0-255)
    B_Motor(LOW,80);  //B 电机反转(默认 LOW 为反转),速度值 80 (范围 0-255)
    delay(1000);      //延时等待 1000ms
}
```

③ Mixly 2.0 rc3 版本示例 1 （注意速度值范围为 0-255）



④ Mixly 2.0 rc3 版本示例 2 （注意速度值范围为 0-255）

